

[INICIAL] TCArg 2026 Contest #3

A. Hexadecimal's theorem

2 seconds, 256 megabytes

Recently, a chaotic virus Hexadecimal advanced a new theorem which will shake the Universe. She thinks that each Fibonacci number can be represented as sum of three not necessary different Fibonacci numbers.

Let's remember how Fibonacci numbers can be calculated. $F_0 = 0$, $F_1 = 1$, and all the next numbers are $F_i = F_{i-2} + F_{i-1}$.

So, Fibonacci numbers make a sequence of numbers: 0, 1, 1, 2, 3, 5, 8, 13, ...

If you haven't run away from the PC in fear, you have to help the virus. Your task is to divide given Fibonacci number n by three not necessary different Fibonacci numbers or say that it is impossible.

Input
The input contains of a single integer n ($0 \leq n < 10^9$) — the number that should be represented by the rules described above. It is guaranteed that n is a Fibonacci number.

Output
Output three required numbers: a , b and c . If there is no answer for the test you have to print "I'm too stupid to solve this problem" without the quotes.

If there are multiple answers, print any of them.

input
3
output
1 1 1

input
13
output
2 3 8

B. Display The Number

1 second, 256 megabytes

You have a large electronic screen which can display up to 998244353 decimal digits. The digits are displayed in the same way as on different electronic alarm clocks: each place for a digit consists of 7 segments which can be turned on and off to compose different digits. The following picture describes how you can display all 10 decimal digits:



As you can see, different digits may require different number of segments to be turned on. For example, if you want to display 1, you have to turn on 2 segments of the screen, and if you want to display 8, all 7 segments of some place to display a digit should be turned on.

You want to display a really large integer on the screen. Unfortunately, the screen is bugged: no more than n segments can be turned on simultaneously. So now you wonder what is the greatest integer that can be displayed by turning on no more than n segments.

Your program should be able to process t different test cases.

Input
The first line contains one integer t ($1 \leq t \leq 100$) — the number of test cases in the input.

Then the test cases follow, each of them is represented by a separate line containing one integer n ($2 \leq n \leq 10^5$) — the maximum number of segments that can be turned on in the corresponding testcase.

It is guaranteed that the sum of n over all test cases in the input does not exceed 10^5 .

Output
For each test case, print the greatest integer that can be displayed by turning on no more than n segments of the screen. Note that the answer may not fit in the standard 32-bit or 64-bit integral data type.

input
2 3 4
output
7 11

C. Maze

2 seconds, 256 megabytes

Pavel loves grid mazes. A grid maze is an $n \times m$ rectangle maze where each cell is either empty, or is a wall. You can go from one cell to another only if both cells are empty and have a common side.

Pavel drew a grid maze with all empty cells forming a connected area. That is, you can go from any empty cell to any other one. Pavel doesn't like it when his maze has too little walls. He wants to turn exactly k empty cells into walls so that all the remaining cells still formed a connected area. Help him.

Input
The first line contains three integers n, m, k ($1 \leq n, m \leq 500, 0 \leq k < s$), where n and m are the maze's height and width, correspondingly, k is the number of walls Pavel wants to add and letter s represents the number of empty cells in the original maze.

Each of the next n lines contains m characters. They describe the original maze. If a character on a line equals ".", then the corresponding cell is empty and if the character equals "#", then the cell is a wall.

Output
Print n lines containing m characters each: the new maze that fits Pavel's requirements. Mark the empty cells that you transformed into walls as "X", the other cells must be left without changes (that is, "." and "#").

It is guaranteed that a solution exists. If there are multiple solutions you can output any of them.

input
3 4 2 #..# ..#. #...
output
#.X# X.#. #...

input
5 4 5 #... #.#. .#.. ...# .#.#
output
#XXX #X#. X#.. ...# .#.#

D. Two Pizzas

4 seconds, 256 megabytes

V. Köln

Guillermo Lago, Ciudades



Alice and Bob are tired of listening to records all day, so they decided to get food. Of course there is no better place to get food than at the Unlimited Topping Pizza Cafe (UTPC).

At the UTPC, every pizza is represented by a set of toppings arranged in a circle. Specifically, there are n positions on the pizza, numbered from 1 to n . Each topping has a tastiness value in the range $[1, 100]$, and each position on a pizza may contain up to two toppings (possibly none). It is well-known that if two toppings line on the same position of a pizza, they will no longer have any tastiness, so the tastiness of a pizza is defined as the sum of tastiness of all toppings on the pizza which do not share a position with any other topping.

This is both Alice's and Bob's first time visiting the UTPC, and they have a dilemma: They each have a pizza in mind (which contains **at most one** topping at each position), but can only afford to purchase one (the pizzas here are quite expensive). You, the chef, know how to solve the dilemma: by merging the two pizzas into one! First, you will rotate Bob's pizza by some amount so that each position in Bob's pizza is aligned with some position in Alice's pizza. Then, you will transfer all of Bob's toppings to the corresponding position on Alice's pizza. Since you want Alice and Bob to return in the future, you must find the maximum tastiness which a pizza can contain after merging Alice's and Bob's pizzas.

Input

The first line of input consists of a single integer n ($1 \leq n \leq 2 \cdot 10^5$) — the number of positions on each pizza.

The second line of input consists of n space-separated integers $a_1, \dots, a_n$ ($0 \leq a_i \leq 100$) — the toppings on Alice's pizza. If $a_i = 0$, then there is no topping at position i . Otherwise, a_i represents the tastiness of the topping at position i .

Similarly, the third line consists of n space-separated integers $b_1, \dots, b_n$ ($0 \leq b_i \leq 100$) — the toppings on Bob's pizza. If $b_i = 0$, then there is no topping at position i . Otherwise, b_i represents the tastiness of the topping at position i .

Output

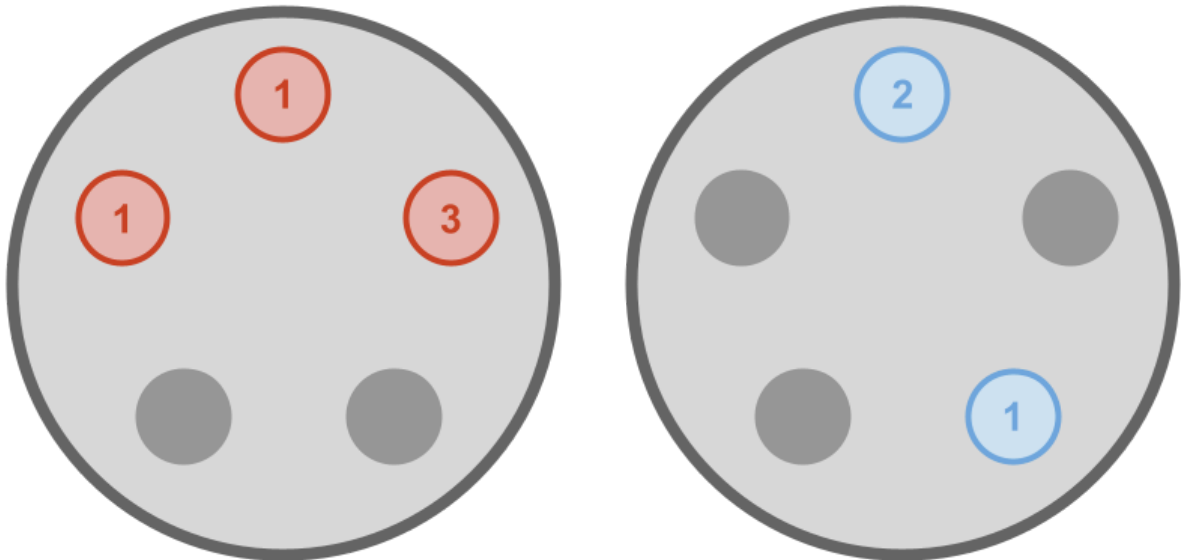
The output consists of a single integer — the maximum possible tastiness of the merged pizza.

input
5 1 3 0 0 1 2 0 1 0 0
output
6

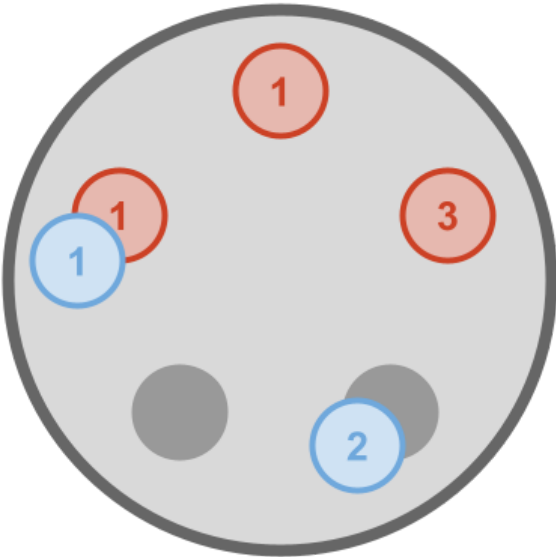
input
8 1 4 11 3 0 0 0 1 2 0 0 0 6 0 5 3
output
29

input
3 1 1 1 1 1 1
output
0

In the first sample, Alice and Bob have the following two pizzas in mind:



We can obtain a pizza with a tastiness of 6 by rotating Bob's pizza twice clockwise relative to Alice's pizza as seen below:



The tastiness of this pizza is $1 + 3 + 2 = 6$ since that is the sum of the toppings which are in a position of their own.

E. Restoring Three Numbers

1 second, 256 megabytes

Polycarp has guessed three positive integers a , b and c . He keeps these numbers in secret, but he writes down four numbers on a board in arbitrary order — their pairwise sums (three numbers) and sum of all three numbers (one number). So, there are four numbers on a board in random order: $a + b$, $a + c$, $b + c$ and $a + b + c$.

You have to guess three numbers a , b and c using given numbers. Print three guessed integers in any order.

Pay attention that some given numbers a , b and c can be equal (it is also possible that $a = b = c$).

Input

The only line of the input contains four positive integers x_1, x_2, x_3, x_4 ($2 \leq x_i \leq 10^9$) — numbers written on a board in random order. It is guaranteed that the answer exists for the given number x_1, x_2, x_3, x_4 .

Output

Print such positive integers a , b and c that four numbers written on a board are values $a + b$, $a + c$, $b + c$ and $a + b + c$ written in some order. Print a , b and c in any order. If there are several answers, you can print any. It is guaranteed that the answer exists.

input
3 6 5 4
output
2 1 3

input
40 40 40 60
output
20 20 20

input
201 101 101 200
output
1 100 100

F. Sample Heat

1 second, 256 megabytes

Blessing It -remix

Nujabes ft. Substantial and Pase Rock, Metaphorical Music



As a music producer, Mai has been trying to get her career off the ground! She has n samples on a record disk. The disk is represented as a string s of n characters, where each sample is labeled with some lowercase character 'a' - 'z'. As part of creating a beat, Mai will pick some substring of samples to use as the foundation of their beat. Mai prefers sounds that have a symmetric sound to them, so the substring she extracts must be a palindrome.

The *heat* of a substring of samples is the sum of the IDs used, where 'a' corresponds to a value of 1, 'b' corresponds to a value of 2, and so forth. If an ID appears multiple times, it is counted multiple times in this sum. Mai wants to compute how valuable the record is, so she wants to compute the sum of the heats of all **distinct** valid substring of samples. Please help her do so!

A palindrome is a string that reads the same forwards and backwards. For example, "aa" and "abacaba" are both palindromes, while "abb" and "utpc" are not.

Two substring are distinct if they have different lengths or at least one position differs between the two substring.

Input

The first line contains a single integer n ($1 \leq n \leq 10^5$) — the number of samples.
The second line contains a single string s of length n , comprising of only the characters 'a' - 'z' — the representation of the samples on the disk.

Output

Output a single integer — the sum of the heats of each distinct palindromic substring.

input
5 aabaa
output
15

For the input "aabaa", the heats of the palindromic substrings are:

- "a" with a heat of 1
- "b" with a heat of 2
- "aa" with a heat of 2
- "aba" with a heat of 4
- "aabaa" with a heat of 6

G. Gambling Nim

2 seconds, 512 megabytes

As you know, the game of "Nim" is played with n piles of stones, where the i -th pile initially contains a_i stones. Two players alternate the turns. During a turn a player picks any non-empty pile and removes any positive number of stones from it. The one who is not able to make a move loses the game.

Petya and Vasya are tired of playing Nim, so they invented their own version of the game and named it the "Gambling Nim". They have n two-sided cards, one side of the i -th card has number a_i written on it, while the other side has number b_i . At the beginning of the game the players put all the cards on the table, each card only one of its sides up, and this side is chosen independently and uniformly. Thus they obtain a sequence $c_1, c_2, ..., c_n$, where c_i is equal to a_i or b_i . Then they take n piles of stones, with i -th pile containing exactly c_i stones and play Nim. Petya takes the first turn.

Given that both players play optimally, find the probability of Petya's victory. Output the answer as an irreducible fraction.

Input

The first line of the input contains a single integer n ($1 \leq n \leq 500\,000$) — the number of cards in the deck.
Each of the following n lines contains the description of one card, consisting of two integers a_i and b_i ($0 \leq a_i, b_i \leq 10^{18}$).

Output

Output the answer as an irreducible fraction p/q . If the probability of Petya's victory is 0, print 0/1.

input
2 1 1 1 1

output
0/1

input
2 1 2 1 2
output
1/2

input
3 0 4 1 5 2 3
output
1/1

H. Lexicographically Minimal Shortest Path

2 seconds, 256 megabytes

You are given a connected undirected unweighted graph with n vertices and m edges. The graph does not contain self-loops and parallel edges. Additionally, on each edge (u_i, v_i) the letter c_i is written.

You have to find the shortest path from the vertex 1 to the vertex n , and if there are several such paths, to find the one which has the string formed by the letters of this path to be lexicographically minimal.

Input

The first line contains two integers n and m ($2 \leq n \leq 200000, 1 \leq m \leq 200000$) — the number of vertices and the number of edges in the graph.

Each of the next m lines contains two integers u_i, v_i ($1 \leq u_i, v_i \leq n, u_i \neq v_i$) — the vertices connected by the i -th edge, and the lowercase Latin letter c_i written on this edge.

Output

In the first line output the integer k — the length of the shortest path from the vertex 1 to the vertex n .

In the second line output $(k + 1)$ integers — the sequence of vertices in the shortest path.

In the third line output the string with k characters — the sequence of letters on this path. This string must be lexicographically minimal among all strings formed by shortest paths from vertex 1 to vertex n .

If there are several possible answers, output any of them.

input
3 2 1 2 a 2 3 b
output
2 1 2 3 ab

input
3 3 1 3 z 1 2 a 2 3 b
output
1 1 3 z

input
4 4 1 2 b 2 4 a 1 3 a 3 4 z

output

```
2
1 3 4
az
```

I. LCM Problem

2 seconds, 256 megabytes

Let $LCM(x, y)$ be the minimum positive integer that is divisible by both x and y . For example, $LCM(13, 37) = 481$, $LCM(9, 6) = 18$.

You are given two integers l and r . Find two integers x and y such that $l \leq x < y \leq r$ and $l \leq LCM(x, y) \leq r$.

Input

The first line contains one integer t ($1 \leq t \leq 10000$) — the number of test cases.

Each test case is represented by one line containing two integers l and r ($1 \leq l < r \leq 10^9$).

Output

For each test case, print two integers:

- if it is impossible to find integers x and y meeting the constraints in the statement, print two integers equal to -1 ;
- otherwise, print the values of x and y (if there are multiple valid answers, you may print any of them).

input

```
4
1 1337
13 69
2 4
88 89
```

output

```
6 7
14 21
2 4
-1 -1
```

J. Tecuci

2 seconds, 512 megabytes



Little IR12660 arrived in Tecuci, land of legendary mustard sauce. He is currently at a mustard fair, where various mustard sauces are being presented. While the mustard sauces were spinning around on a conveyor belt, he came up with the following idea:

You are given an array A of length n and an integer k . You are allowed to perform at most k cyclic shifts on any subarray of A .

Determine the maximum sum of a contiguous subarray that can be obtained after performing at most k such cyclic shifts.

A **cyclic shift** in any direction on a subarray means:

- For a **right cyclic shift**, the last element of the subarray is moved to the front, and all other elements are shifted one position to the right.
 - Example: For $A = [1, 2, 3, 4, 5]$, a right cyclic shift on the subarray $[2, 3, 4]$ (indices 2 to 4) results in $A = [1, 4, 2, 3, 5]$.
- For a **left cyclic shift**, the first element of the subarray is moved to the back, and all other elements are shifted one position to the left.
 - Example: For $A = [1, 2, 3, 4, 5]$, a left cyclic shift on the subarray $[2, 3, 4]$ (indices 2 to 4) results in $A = [1, 3, 4, 2, 5]$.

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

Each test case consists of:

- A single line containing two integers n and k ($1 \leq n \leq 10^6$, $k \geq 0$, $n \cdot k \leq 10^6$) — the size of the array and the maximum number of allowed cyclic shifts.
- A second line containing n integers $a_1, a_2, \dots, a_n$ ($-10^9 \leq a_i \leq 10^9$) — the elements of the array.

It is guaranteed that the sum of $n \cdot k$ across all testcases does not exceed 10^6 and the sum of n across all testcases does not exceed 10^6 .

Output

For each test case, output a single integer — the maximum sum of a contiguous subarray that can be obtained after performing at most k cyclic shifts.

input
<pre> 4 5 2 -1 2 -3 4 -5 7 3 4 -2 1 3 -1 -3 2 6 1 -1 -1 2 -1 -1 -1 5 5 -1 -1 -1 -1 -1 </pre>
output
<pre> 6 10 2 -1 </pre>

K. Quests

2.5 seconds, 256 megabytes

Monocarp is playing a computer game. In order to level up his character, he can complete quests. There are n quests in the game, numbered from 1 to n .

Monocarp can complete quests according to the following rules:

- the 1-st quest is always available for completion;
- the i -th quest is available for completion if all quests $j < i$ have been completed at least once.

Note that Monocarp can complete the same quest multiple times.

For each completion, the character gets some amount of experience points:

- for the first completion of the i -th quest, he gets a_i experience points;
- for each subsequent completion of the i -th quest, he gets b_i experience points.

Monocarp is a very busy person, so he has free time to complete no more than k quests. Your task is to calculate the maximum possible total experience Monocarp can get if he can complete no more than k quests.

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains two integers n and k ($1 \leq n \leq 2 \cdot 10^5$; $1 \leq k \leq 2 \cdot 10^5$) — the number of quests and the maximum number of quests Monocarp can complete, respectively.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^3$).

The third line contains n integers $b_1, b_2, \dots, b_n$ ($1 \leq b_i \leq 10^3$).

Additional constraint on the input: the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print a single integer — the maximum possible total experience Monocarp can get if he can complete no more than k quests.

input
<pre> 4 4 7 4 3 1 2 1 1 1 1 3 2 1 2 5 3 1 8 5 5 3 2 4 1 4 2 3 1 4 7 6 4 1 4 5 4 5 10 1 5 1 2 5 1 </pre>

output

13
4
15
15

In the first test case, one of the possible quest completion sequences is as follows: 1, 1, 2, 3, 2, 4, 4; its total experience is equal to $\underline{4} + 1 + \underline{3} + \underline{1} + 1 + \underline{2} + 1 = 13$ (the underlined numbers correspond to the instances when we complete a quest for the first time).

In the second test case, one of the possible quest completion sequences is as follows: 1, 1; its total experience is equal to $\underline{1} + 3 = 4$.

In the third test case, one of the possible quest completion sequences is as follows: 1, 2, 2, 2, 3; its total experience is equal to $\underline{3} + \underline{2} + 3 + 3 + \underline{4} = 15$.

L. Kefa and First Steps

2 seconds, 256 megabytes

Kefa decided to make some money doing business on the Internet for exactly n days. He knows that on the i -th day ($1 \leq i \leq n$) he makes a_i money. Kefa loves progress, that's why he wants to know the length of the maximum non-decreasing subsegment in sequence a_i . Let us remind you that the subsegment of the sequence is its continuous fragment. A subsegment of numbers is called non-decreasing if all numbers in it follow in the non-decreasing order.

Help Kefa cope with this task!

Input

The first line contains integer n ($1 \leq n \leq 10^5$).

The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$).

Output

Print a single integer — the length of the maximum non-decreasing subsegment of sequence a .

input

6
2 2 1 3 4 1

output

3

input

3
2 2 9

output

3

In the first test the maximum non-decreasing subsegment is the numbers from the third to the fifth one.

In the second test the maximum non-decreasing subsegment is the numbers from the first to the third one.

M. Number of Pairs

2 seconds, 256 megabytes

You are given an array a of n integers. Find the number of pairs (i, j) ($1 \leq i < j \leq n$) where the sum of $a_i + a_j$ is greater than or equal to l and less than or equal to r (that is, $l \leq a_i + a_j \leq r$).

For example, if $n = 3$, $a = [5, 1, 2]$, $l = 4$ and $r = 7$, then two pairs are suitable:

- $i = 1$ and $j = 2$ ($4 \leq 5 + 1 \leq 7$);
- $i = 1$ and $j = 3$ ($4 \leq 5 + 2 \leq 7$).

Input

The first line contains an integer t ($1 \leq t \leq 10^4$). Then t test cases follow.

The first line of each test case contains three integers n, l, r ($1 \leq n \leq 2 \cdot 10^5$, $1 \leq l \leq r \leq 10^9$) — the length of the array and the limits on the sum in the pair.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$).

It is guaranteed that the sum of n overall test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output a single integer — the number of index pairs (i, j) ($i < j$), such that $l \leq a_i + a_j \leq r$.

input
<pre> 4 3 4 7 5 1 2 5 5 8 5 1 2 4 3 4 100 1000 1 1 1 1 5 9 13 2 5 5 1 1 </pre>
output
<pre> 2 7 0 1 </pre>

N. Little Elephant and Problem

2 seconds, 256 megabytes

The Little Elephant has got a problem — somebody has been touching his sorted by non-decreasing array a of length n and possibly swapped some elements of the array.

The Little Elephant doesn't want to call the police until he understands if he could have accidentally changed the array himself. He thinks that he could have accidentally changed array a , only if array a can be sorted in no more than one operation of swapping elements (not necessarily adjacent). That is, the Little Elephant could have accidentally swapped some two elements.

Help the Little Elephant, determine if he could have accidentally changed the array a , sorted by non-decreasing, himself.

Input

The first line contains a single integer n ($2 \leq n \leq 10^5$) — the size of array a . The next line contains n positive integers, separated by single spaces and not exceeding 10^9 , — array a .

Note that the elements of the array are not necessarily distinct numbers.

Output

In a single line print "YES" (without the quotes) if the Little Elephant could have accidentally changed the array himself, and "NO" (without the quotes) otherwise.

input
<pre> 2 1 2 </pre>
output
<pre> YES </pre>

input
<pre> 3 3 2 1 </pre>
output
<pre> YES </pre>

input
<pre> 4 4 3 2 1 </pre>
output
<pre> NO </pre>

In the first sample the array has already been sorted, so to sort it, we need 0 swap operations, that is not more than 1. Thus, the answer is "YES".

In the second sample we can sort the array if we swap elements 1 and 3, so we need 1 swap operation to sort the array. Thus, the answer is "YES".

In the third sample we can't sort the array in more than one swap operation, so the answer is "NO".